FADENA

LICENCIATURA EN CIBERDEFENSA

DISPOSITIVOS REMOTOS E INTERNET DE LAS COSAS
CÁTEDRA: DIEGO RACERO
AÑO: 2026

PROTOCOLOS IoT

Protocolos IoT: HTTP, HTTPS, MQTT, QoS y Seguridad

En esta sección vamos a describir en detalle varios de los protocolos de los que estuvimos hablando en ocasiones anteriores.

HTTP en IoT

¿Qué es HTTP?

HTTP (Hypertext Transfer Protocol) es un protocolo de capa de aplicación desarrollado por Tim Berners-Lee en CERN. Aunque fue diseñado originalmente para la web, se utiliza extensamente en IoT para la comunicación máquina-a-máquina (M2M).

Características Principales

Modelo Request-Response:

- El cliente envía solicitudes HTTP al servidor
- El servidor procesa y responde con datos o códigos de estado
- Protocolo sin estado (stateless) por defecto

Métodos HTTP en IoT:

- **GET**: Solicitar datos de sensores o estado de dispositivos
- **POST**: Enviar datos de telemetría al servidor
- **PUT**: Actualizar configuración de dispositivos
- **DELETE**: Eliminar recursos

Códigos de Estado Comunes:

- `200 OK`: Solicitud exitosa
- `201 Created`: Recurso creado exitosamente
- `400 Bad Request`: Error en la solicitud
- `401 Unauthorized`: Autenticación requerida
- `404 Not Found`: Recurso no encontrado
- `500 Internal Server Error`: Error del servidor

Uso en IoT

Ventajas:

- Ampliamente soportado y conocido
- Integración fácil con APIs REST
- Compatible con infraestructura web existente
- HTTP/2 ofrece multiplexación y compresión de headers

Desventajas:

- Mayor overhead comparado con protocolos IoT específicos
- No optimizado para dispositivos con recursos limitados
- Consume más ancho de banda que otros protocolos usados en IoT

- Batería y energía: no ideal para dispositivos de bajo consumo
- Conexiones cortas: requiere establecer conexión para cada solicitud

Casos de Uso

- Impresión 3D en red (manufacturing aditivo)
- Actualizaciones de firmware
- Dispositivos con conexión constante a energía
- Aplicaciones donde el volumen de datos es alto
- Integración con servicios web y APIs

HTTPS en IoT

¿Qué es HTTPS?

HTTPS (HTTP Secure) es la versión segura de HTTP que utiliza TLS/SSL para cifrar las comunicaciones entre cliente y servidor. Es crucial para proteger datos sensibles en aplicaciones IoT.

Características de Seguridad

HTTPS (Hypertext Transfer Protocol Secure) constituye la versión segura del protocolo estándar HTTP. Su arquitectura se fundamenta en la implementación de protocolos criptográficos como TLS/SSL (Transport Layer Security / Secure Sockets Layer), los cuales se encargan de cifrar de extremo a extremo las comunicaciones entre el cliente (por ejemplo, un gateway edge) y el servidor en la nube.

En el contexto de las aplicaciones de Internet de las Cosas (IoT), y particularmente en el despliegue de infraestructuras para la agricultura de precisión, su implementación resulta de carácter crítico por las siguientes razones:

- **Confidencialidad de la información:** Garantiza que los datos sensibles, tales como lecturas de parámetros edáficos, coordenadas GPS del equipamiento o información productiva procesada localmente, viajen de forma cifrada. Esto previene la intercepción y el espionaje industrial.

- **Autenticación e integridad:** A través del uso de certificados digitales bidireccionales, HTTPS asegura que el dispositivo edge se esté comunicando genuinamente con el servidor central autorizado. Esto mitiga significativamente el riesgo de ataques de intermediario (Man-in-the-Middle), donde un actor malicioso podría alterar los paquetes de datos en tránsito.

- **Protección de comandos críticos:** Las instrucciones operativas que descienden desde la nube hacia los actuadores en el campo (por ejemplo, la orden de modificar la dosificación de agroquímicos en un sistema de riego) requieren obligatoriamente de este canal seguro. Cualquier manipulación externa de estos comandos podría desencadenar consecuencias severas para la integridad de los cultivos y la rentabilidad de la explotación.

MQTT - Protocolo Principal para IoT

Introducción a MQTT

MQTT (Message Queuing Telemetry Transport) es el protocolo estándar de facto para IoT, estandarizado por OASIS e ISO. Fue diseñado específicamente para comunicaciones M2M con ancho de banda limitado y conexiones no confiables.

Arquitectura Publish-Subscribe

Componentes Principales:

1. **Publisher (Publicador)**: Dispositivo que envía mensajes
2. **Subscriber (Suscriptor)**: Dispositivo que recibe mensajes
3. **Broker**: Servidor central que gestiona y enruta mensajes
4. **Topic**: Jerarquía de categorías para organizar mensajes

Modelo de Comunicación:

Publisher → [Topic] → Broker → [Topic] → Subscriber(s)

Desacoplamiento:

- Los publicadores y suscriptores no se conocen entre sí
- La comunicación pasa a través del broker
- Permite escalabilidad y flexibilidad

Topics y Wildcards

Estructura de Topics:

casa/sala/temperatura
industria/linea1/robot01/status
ciudad/zona-norte/sensores/aire

Separadores: Se usa `/` para crear jerarquías

Wildcards:

- `+` (single-level): Coincide con un nivel exacto
  - `casa/+/temperatura` → casa/sala/temperatura, casa/cocina/temperatura
- `#` (multi-level): Coincide con múltiples niveles
  - `casa/#` → casa/sala/temperatura, casa/cocina/luz/estado

Características Principales

Ligero y Eficiente:

- Mensajes pequeños (mínimo 2 bytes de header)

- Protocolo binario optimizado
- Consume hasta 80% menos ancho de banda que HTTP
- Ideal para conexiones de baja velocidad

Confiabilidad:

- Tres niveles de Quality of Service (QoS 0, 1, 2)
- Sesiones persistentes
- Last Will and Testament (LWT)
- Retained messages

Escalabilidad:

- Soporta millones de dispositivos conectados
- Modelo publish-subscribe eficiente
- Bajo uso de recursos en clientes

Puertos MQTT

- **Puerto 1883**: MQTT sin cifrado (TCP)
- **Puerto 8883**: MQTT con TLS/SSL
- **Puerto 80/443**: MQTT sobre WebSockets

Funcionalidades Avanzadas

1. Sesiones Persistentes:

- El broker recuerda suscripciones del cliente
- Almacena mensajes no entregados (según QoS)
- Reconexión rápida tras desconexión

2. Retained Messages:

- El broker guarda el último mensaje de un topic
- Nuevos suscriptores reciben inmediatamente este mensaje
- Útil para estados y configuraciones

3. Last Will and Testament (LWT):

- Mensaje enviado por el broker si el cliente se desconecta inesperadamente
- Permite detectar fallos de dispositivos
- Implementa mecanismos de heartbeat

4. Keep-Alive:

- Intervalo de ping entre cliente y broker
- Mantiene la conexión activa
- Detecta desconexiones silenciosas

Versiones MQTT

- **MQTT 3.1**: Versión original
- **MQTT 3.1.1**: Versión más común y estable

- **MQTT 5.0**: Versión más reciente con mejoras:
  - Propiedades de usuario personalizadas
  - Razones de desconexión mejoradas
  - Topic aliases
  - Shared subscriptions

Casos de Uso

- **Smart Home**: Control de iluminación, termostatos, cámaras
- **Automatización Industrial (IIoT)**: Monitoreo de máquinas, sensores
- **Agricultura**: Sensores de humedad del suelo, estaciones meteorológicas
- **Ciudades Inteligentes**: Alumbrado público, gestión de tráfico
- **Salud**: Dispositivos médicos wearables, monitoreo remoto

Quality of Service (QoS) en MQTT

Introducción al QoS

El Quality of Service en MQTT define el nivel de garantía para la entrega de mensajes entre publicador y suscriptor. Es una característica crítica que diferencia MQTT de otros protocolos.

Niveles de QoS

QoS 0: At Most Once (Como máximo una vez)

Características:

- Modelo "fire and forget" (disparar y olvidar)
- Sin confirmación de recepción
- Sin almacenamiento de mensajes
- Overhead mínimo

Funcionamiento:

Publisher → PUBLISH → Broker → PUBLISH → Subscriber

Ventajas:

- Máxima velocidad de transmisión
- Menor consumo de recursos
- Menor latencia

Desventajas:

- Posible pérdida de mensajes
- Sin garantía de entrega

Casos de Uso:

- Lecturas de sensores frecuentes donde pérdida ocasional es aceptable
- Datos ambientales no críticos (temperatura, humedad)
- Actualizaciones de estado frecuentes
- Telemetría de alta frecuencia

QoS 1: At Least Once (Al menos una vez)

Características:

- Garantiza entrega del mensaje
- Requiere confirmación (PUBACK)
- Posibles duplicados
- Overhead moderado

Funcionamiento:

Publisher → PUBLISH → Broker → PUBACK → Publisher

↓

Broker → PUBLISH → Subscriber → PUBACK → Broker

Ventajas:

- Garantía de entrega
- Balance entre confiabilidad y rendimiento
- Overhead aceptable

Desventajas:

- Posibles mensajes duplicados
- Mayor latencia que QoS 0
- Requiere lógica de deduplicación en aplicación

Casos de Uso:

- Comandos de control de dispositivos
- Datos importantes que deben llegar
- Actualizaciones de configuración
- Alertas y notificaciones

QoS 2: Exactly Once (Exactamente una vez)

Características:

- Garantiza entrega única
- Handshake de 4 pasos
- Sin duplicados ni pérdidas
- Mayor overhead

Funcionamiento:

Publisher → PUBLISH → Broker → PUBREC → Publisher

↓

Publisher ← PUBCOMP ← Broker ← PUBREL ← Publisher

Broker → PUBLISH → Subscriber → PUBREC → Broker

↓

Broker ← PUBCOMP ← Subscriber ← PUBREL ← Broker

Ventajas:

- Máxima confiabilidad
- Garantía de no duplicación
- Ideal para datos críticos

Desventajas:

- Mayor latencia (4 pasos)
- Mayor consumo de recursos
- Más overhead de red

Casos de Uso:

- Transacciones financieras
- Comandos críticos de seguridad
- Control de maquinaria industrial crítica
- Sistemas de alarma
- Registros de auditoría

Downgrade de QoS

El QoS efectivo es el menor entre:

1. QoS del mensaje publicado
2. QoS de la suscripción

Ejemplo:

Publicador publica con QoS 2
Suscriptor está suscrito con QoS 1

→ El mensaje se entrega con QoS 1

Comparación de QoS

Característica	QoS 0	QoS 1	QoS 2
Garantía	Ninguna	Al menos una vez	Exactamente una vez

Duplicados	Posible pérdida	Posibles	No
Overhead	Mínimo	Moderado	Alto
Latencia	Más baja	Media	Más alta
Pasos	1	2	4
Uso de red	Mínimo	Moderado	Alto
Complejidad	Baja	Media	Alta

Selección del QoS Apropiado

QoS 0:

- Datos no críticos
- Alta frecuencia de actualización
- Recursos muy limitados

QoS 1:

- Mayoría de aplicaciones IoT
- Balance óptimo rendimiento/confiabilidad
- Datos importantes pero tolerantes a duplicados

QoS 2:

- Aplicaciones críticas
- Datos financieros o de facturación
- Comandos que no pueden ejecutarse dos veces

Seguridad en Protocolos IoT

Importancia de la Seguridad en IoT

Los dispositivos IoT son objetivos atractivos para atacantes debido a:

- Exposición física en campo
- Actualizaciones poco frecuentes
- Recursos limitados para seguridad
- Gran número de dispositivos vulnerables

Seguridad en MQTT

Transport Layer Security (TLS/SSL)

Cifrado de Transporte:

- Protege datos en tránsito
- Previene espionaje (eavesdropping)
- Evita ataques Man-in-the-Middle
- Garantiza integridad de mensajes

Versiones TLS Recomendadas:

- **TLS 1.2**: Mínimo aceptable
- **TLS 1.3**: Recomendado (más rápido y seguro)
- **Evitar**: TLS 1.0 y 1.1 (obsoletos)

Mejores Prácticas:

- Usar contraseñas largas y complejas
- Cambiar contraseñas periódicamente
- Nunca hardcodear contraseñas
- Usar variables de entorno o almacenamiento seguro

Autenticación con Certificados X.509:

Ventajas:

- Más seguro que usuario/contraseña
- Autenticación mutua (cliente y servidor)
- Desconexión en fase de handshake si falla
- Difícil de comprometer

Gestión de Certificados:

- Usar una Certificate Authority (CA) propia o de confianza
- Implementar rotación de certificados
- Mantener claves privadas seguras
- Usar hardware security modules (HSM) cuando sea posible

Control de Acceso Basado en Atributos (ABAC):

- Políticas dinámicas basadas en atributos del cliente
- Mayor flexibilidad que ACLs
- Soportado en brokers enterprise

Seguridad de Red

Firewalls:

- Permitir solo puertos necesarios (1883, 8883)
- Implementar reglas de entrada/salida
- Usar network segmentation

VPN:

- Túneles VPN para conectar dispositivos remotos
- Añade capa adicional de cifrado
- Protege todo el tráfico, no solo MQTT

Network Isolation:

- Separar red IoT de red corporativa
- Usar VLANs para segmentar dispositivos
- Implementar microsegmentación

Mejores Prácticas de Seguridad

Principio de Mínimo Privilegio:

- Otorgar solo permisos necesarios
- Un topic por función/dispositivo
- Limitar wildcards en suscripciones

Rate Limiting:

- Limitar frecuencia de publicación
- Prevenir ataques DDoS

- Configurar en el broker

Actualización Regular:

- Mantener firmware actualizado
- Actualizar bibliotecas y dependencias
- Parchear seguridad por vulnerabilidades conocidas
- Implementar OTA (Over-The-Air) updates seguras

Almacenamiento Seguro:

- Cifrar credenciales en dispositivos
- Usar secure elements o TPM
- No almacenar claves en texto plano

Comparación de Protocolos

HTTP vs MQTT

Característica	HTTP/HTTPS	MQTT
Modelo	Request-Response	Publish-Subscribe
Transporte	TCP	TCP

Overhead	Alto	Bajo
QoS	No nativo	3 niveles (0,1,2)
Persistencia	No	Sí (sesiones)
Complejidad	Media	Media
Seguridad	TLS/SSL	TLS/SSL
Uso de batería	Alto	Medio
Mejor para	APIs, Web Services	IoT general, IIoT
Ancho de banda	Alto	Bajo (80% menos que HTTP)

Cuándo Usar Cada Protocolo

HTTP/HTTPS:

- Integración con servicios web existentes
- APIs RESTful
- Dispositivos con conexión constante a energía
- Volumen de datos alto
- Actualizaciones de firmware
- Compatibilidad con infraestructura web

MQTT:

- Comunicación M2M en general
- Dispositivos con recursos limitados
- Conexiones no confiables (celular, WiFi inestable)
- Necesidad de QoS garantizado
- Arquitectura escalable (millones de dispositivos)
- Smart home, IIoT, telemetría

Combinación de Protocolos

Muchas arquitecturas IoT usan múltiples protocolos:

[Sensores] --MQTT--> [Gateway] --HTTPS--> [Cloud Platform]
↓
[Base de datos]

↓

[Analytics/ML]

Brokers MQTT Públicos para Testing

1. HiveMQ Public Broker

Información:

- Broker público gratuito de HiveMQ
- Ubicación: Frankfurt, Alemania (AWS)
- Sin registro requerido

Detalles de Conexión:

Host: broker.hivemq.com
Puerto TCP: 1883
Puerto WebSocket: 8000
Puerto TLS: 8883

Cliente Web:

- https://www.hivemq.com/mqtt/public-mqtt-broker/
- Cliente WebSocket integrado en el navegador

2. Eclipse Mosquitto Test Server

Información:

- Servidor público de la comunidad Mosquitto
- Soporta múltiples puertos y configuraciones

Detalles de Conexión:

Host: test.mosquitto.org
Puerto TCP: 1883
Puerto TLS: 8883, 8884
Puerto WebSocket: 8080
Puerto WebSocket TLS: 8081

Características:

- Certificados Let's Encrypt para TLS
- Sin autenticación requerida (puerto 1883)
- https://test.mosquitto.org/

3. EMQX Public Broker

Información:

- Broker MQTT 5.0 público
- Ubicación: Varios servidores globales

Detalles de Conexión:

Host: broker.emqx.io (Global)
Host: broker-cn.emqx.io (China)
Puerto TCP: 1883
Puerto TLS: 8883
Puerto WebSocket: 8083
Puerto WebSocket TLS: 8084

Acceso:

- https://www.emqx.com/en/mqtt/public-mqtt5-broker

Comparación de Brokers Públicos

Broker	Ubicación	MQTT 5.0	TLS	WebSocket	Estabilidad
HiveMQ	Alemania	✓	✓	✓	Alta
Mosquitto	Francia	✓	✓	✓	Media-Alta
EMQX	Global	✓	✓	✓	Alta

Nota: Estos brokers son para testing y prototipado. NO usar en producción ni para datos sensibles.

Referencias y Recursos

Documentación Oficial

MQTT:

- Especificación MQTT 3.1.1: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html
- Especificación MQTT 5.0: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- MQTT.org: https://mqtt.org/

HTTP/HTTPS:

- MDN Web Docs HTTP: https://developer.mozilla.org/en-US/docs/Web/HTTP
- RFC 7230 HTTP/1.1: https://tools.ietf.org/html/rfc7230
- RFC 8446 TLS 1.3: https://tools.ietf.org/html/rfc8446

Tutoriales y Guías

MQTT Essentials (HiveMQ):

- Serie completa sobre MQTT
- https://www.hivemq.com/mqtt-essentials/

MQTT Beginner's Guide (EMQ):

- Guía completa para principiantes
- https://www.emqx.com/en/blog/the-easiest-guide-to-getting-started-with-mqtt

MQTT Security Fundamentals:

- https://www.hivemq.com/blog/mqtt-security-fundamentals-tls-ssl/
- https://www.hivemq.com/blog/mqtt-security-fundamentals-authentication-authorization/

Conclusión

Los protocolos HTTP/HTTPS y MQTT son fundamentales en el ecosistema IoT moderno. Mientras HTTP/HTTPS proporciona integración con infraestructura web existente, MQTT se ha establecido como el estándar de facto para comunicaciones IoT eficientes y confiables.

Puntos Clave:

1. **HTTP/HTTPS**: Ideal para APIs, integración web y transferencia de datos voluminosos
2. **MQTT**: Optimizado para IoT con bajo consumo de recursos y modelo publish-subscribe
3. **QoS**: Permite balance entre confiabilidad y rendimiento según necesidades
4. **Seguridad**: Crítica en IoT, requiere implementación en múltiples capas (TLS, autenticación, autorización)
5. **Testing**: Múltiples herramientas disponibles para desarrollo, testing y simulación

Recomendaciones Finales:

- Comenzar con brokers públicos para aprendizaje y prototipos
- Implementar siempre TLS/SSL en producción
- Elegir QoS apropiado para cada caso de uso
- Monitorear y registrar actividad para seguridad
- Mantenerse actualizado con últimas versiones de protocolos y seguridad
- Probar exhaustivamente antes de desplegar en producción

Esta semana se deben comenzar la actividad obligatoria 1, con capturas de HTTP y MQTT. En la semana 6 deberán hacer lo mismo con CoAP y Websocket simulados en Python con FastAPI. Tendrán que clonar el repositorio de Github, usar el generador de código Antigravity para que les explique el código, poner el código a funcionar y hacer capturas con Wireshark.

Vean cuidadosamente los tutoriales que les explican cómo instalar y poner a funcionar todo.